

Investigación MDI

Introducción

En el desarrollo de aplicaciones de escritorio es importante contar con una interfaz que permita organizar de manera adecuada las diferentes funciones del programa. Cuando una aplicación posee varios módulos o formularios, puede resultar poco práctico abrir cada uno como una ventana independiente del sistema operativo.

Una alternativa para solucionar este problema es utilizar la arquitectura MDI (Multiple Document Interface), que permite trabajar con una ventana principal que funciona como contenedor de diferentes ventanas internas. De esta manera, el usuario puede acceder a distintos formularios desde un mismo espacio de trabajo.

En Java, la biblioteca **Swing** proporciona diferentes componentes que permiten implementar este tipo de arquitectura. Entre los más importantes se encuentran `JDesktopPane`, `JInternalFrame`, `JMenuBar`, `JMenu` y `JMenuItem`.

El uso conjunto de estos componentes permite desarrollar aplicaciones de escritorio organizadas, modulares y con una navegación centralizada.

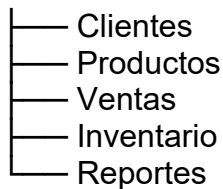
¿Qué es un formulario contenedor (MDI)?

MDI, cuyo significado es **Multiple Document Interface** o **Interfaz de Múltiples Documentos**, es un modelo de interfaz gráfica en el cual existe una ventana principal que funciona como contenedor de diferentes ventanas secundarias.

En este modelo, la ventana principal proporciona un área de trabajo donde pueden abrirse diferentes formularios o documentos. Las ventanas secundarias permanecen dentro de la ventana principal y pueden ser administradas por el usuario.

Por ejemplo, una aplicación podría tener una ventana principal con las siguientes opciones:

Ventana principal



Al seleccionar una opción del menú, el formulario correspondiente se abre dentro de la ventana principal.

Una de las principales características de MDI es que el usuario no necesita tener múltiples ventanas independientes abiertas en el escritorio del sistema operativo. Todo se administra dentro de una ventana principal.

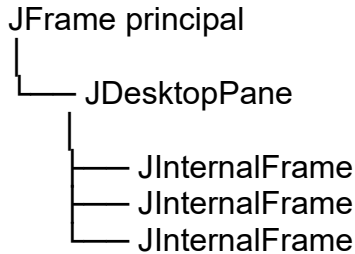
En Java Swing, la arquitectura MDI normalmente se implementa utilizando un `JDesktopPane` como área de trabajo y varios `JInternalFrame` como ventanas internas.

¿Qué es un `JDesktopPane`?

`JDesktopPane` es un componente de Java Swing diseñado específicamente para funcionar como un **escritorio o área de trabajo para ventanas internas**.

Su función principal es contener objetos `JInternalFrame`. Es decir, dentro de un `JDesktopPane` se pueden abrir diferentes formularios internos.

Una representación sencilla sería:



El JDesktopPane permite que las ventanas internas puedan moverse, seleccionarse, minimizarse, maximizarse y cerrarse, dependiendo de las propiedades configuradas en cada JInternalFrame.

Ejemplo básico

```
JDesktopPane desktop = new JDesktopPane();

JInternalFrame ventana =
    new JInternalFrame("Formulario", true, true, true, true);

desktop.add(ventana);

ventana.setVisible(true);
```

En este ejemplo:

- JDesktopPane representa el área de trabajo.
- JInternalFrame representa una ventana interna.
- desktop.add(ventana) agrega la ventana al escritorio.
- setVisible(true) permite mostrarla.

Por lo tanto, JDesktopPane es uno de los componentes fundamentales para construir una aplicación con arquitectura MDI en Java Swing.

¿Qué es un JInternalFrame?

JInternalFrame es un componente de Java Swing que representa una **ventana interna** dentro de un JDesktopPane.

A diferencia de un JFrame, un JInternalFrame no representa normalmente una ventana independiente del sistema operativo. En una arquitectura MDI, se coloca dentro del JDesktopPane.

Un JInternalFrame puede contener diferentes componentes gráficos, como:

- JLabel
- JTextField
- JButton
- JTable

- JComboBox
- JPanel
- Otros componentes de Swing.

Además, puede configurarse para permitir diferentes acciones.

Por ejemplo:

```
JInternalFrame ventana = new JInternalFrame(
    "Clientes",
    true,
    true,
    true,
    true
);
```

Los cuatro valores booleanos indican diferentes propiedades:

true → permite cerrar
 true → permite maximizar
 true → permite minimizar
 true → permite cambiar el tamaño

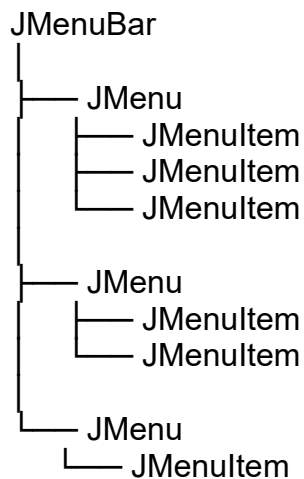
De esta manera se puede crear una ventana interna con características similares a una ventana tradicional, pero dentro del área de trabajo de la aplicación.

¿Cómo funciona un JMenuBar?

JMenuBar es el componente de Swing utilizado para crear la **barra de menús** de una aplicación.

La barra de menú normalmente se encuentra en la parte superior de la ventana principal y contiene diferentes categorías de opciones.

Su estructura puede representarse de la siguiente manera:



Por ejemplo:

Archivo
Edición
Clientes
Productos
Ventas
Ayuda

Cada categoría puede contener diferentes opciones.

Ejemplo

```
JMenuBar barra = new JMenuBar();  
  
JMenu menuArchivo = new JMenu("Archivo");  
  
JMenuItem itemSalir = new JMenuItem("Salir");  
  
menuArchivo.add(itemSalir);  
  
barra.add(menuArchivo);  
  
setJMenuBar(barra);
```

En este ejemplo se crea:

1. Una barra de menú.
2. Un menú llamado "Archivo".
3. Una opción llamada "Salir".
4. La opción se agrega al menú.
5. El menú se agrega a la barra.

Cuando el usuario selecciona un JMenuItem, se puede ejecutar una acción mediante un evento.

¿Cómo abrir formularios hijos desde un menú?

Para abrir un formulario hijo desde un menú se utiliza normalmente un evento asociado al JMenuItem.

El procedimiento general es:

1. El usuario selecciona una opción del menú.
2. Se ejecuta el evento del JMenuItem.
3. Se crea una instancia del formulario.
4. El formulario se agrega al JDesktopPane.
5. Se hace visible el formulario.

Por ejemplo:

```
private void menuClientesActionPerformed(
    java.awt.event.ActionEvent evt) {

    frmClientes ventana = new frmClientes();

    desktopPane.add(ventana);

    ventana.setVisible(true);
}
```

En este caso, frmClientes representa un formulario que hereda de JInternalFrame.

La instrucción:

```
desktopPane.add(ventana);
```

agrega el formulario al área de trabajo.

Después:

```
ventana.setVisible(true);
```

hace que el formulario sea visible para el usuario.

Evitar formularios duplicados

En una aplicación también puede ser conveniente comprobar si el formulario ya está abierto antes de crear una nueva instancia.

Por ejemplo:

```
for (JInternalFrame frame : desktopPane.getAllFrames()) {

    if (frame instanceof frmClientes) {

        try {
            frame.setSelected(true);
        } catch (java.beans.PropertyVetoException e) {
            e.printStackTrace();
        }

        return;
    }
}
```

```
frmClientes ventana = new frmClientes();
```

```
desktopPane.add(ventana);
```

```
ventana.setVisible(true);
```

Con esta técnica se evita abrir varias copias del mismo formulario.

Relación entre JFrame, JDesktopPane y JInternalFrame

Estos componentes cumplen funciones diferentes dentro de una aplicación MDI.

JFrame

Es la ventana principal de la aplicación.

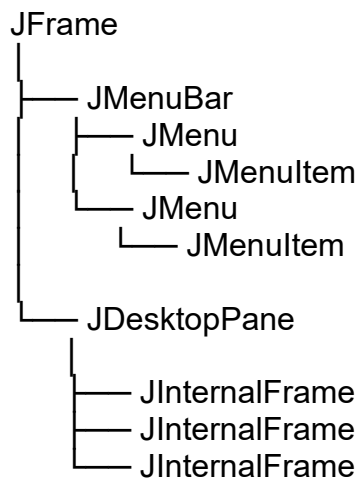
JDesktopPane

Es el área de trabajo que se encuentra dentro de la ventana principal.

JInternalFrame

Representa los formularios o ventanas secundarias que se muestran dentro del JDesktopPane.

La estructura sería:



Esta estructura permite separar la navegación de la aplicación del contenido de cada formulario.

Ventajas de utilizar esta arquitectura en aplicaciones de escritorio

La utilización de una arquitectura MDI presenta diferentes ventajas.

- **Organización**
- **Navegación sencilla**
- **Administración de ventanas**

- **Menor cantidad de ventanas independientes**
- **Modularidad**
- **Facilidad de mantenimiento**
- **Escalabilidad**

Conclusión

La arquitectura MDI es una alternativa para desarrollar aplicaciones de escritorio que contienen múltiples módulos o formularios. Su principal característica es utilizar una ventana principal como contenedor de diferentes ventanas internas.

En Java Swing, el componente `JDesktopPane` permite crear el área de trabajo donde se colocan los formularios, mientras que `JInternalFrame` representa cada una de las ventanas internas.

Por otra parte, `JMenuBar`, `JMenu` y `JMenuItem` permiten crear una estructura de navegación mediante menús. Al asociar eventos a las opciones del menú es posible abrir los formularios correspondientes dentro del `JDesktopPane`.

En conjunto, estos componentes permiten desarrollar interfaces organizadas, modulares y fáciles de utilizar. Además, la separación de los diferentes formularios facilita el mantenimiento y la ampliación de las aplicaciones de escritorio.

Bibliografía

Oracle. (s. f.). *JDesktopPane Class*. Java Platform, Standard Edition API Documentation.
<https://docs.oracle.com/en/java/javase/>

Oracle. (s. f.). *JInternalFrame Class*. Java Platform, Standard Edition API Documentation.
<https://docs.oracle.com/en/java/javase/>

Oracle. (s. f.). *JMenuBar Class*. Java Platform, Standard Edition API Documentation.
<https://docs.oracle.com/en/java/javase/>

Oracle. (s. f.). *How to Use Menus*. The Java Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/components/menu.html>

Oracle. (s. f.). *Creating a GUI With Swing*. The Java Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/>